Paiflow Hackathon Mechanics

Updated Oct 12, 2026

Build an app that solves a real problem with automated payments. Choose your idea and start building now. You may deploy, integrate Paiflow and test payments before the event once your team credentials are available.

At a glance

What	One-day, in-person hackathon to build apps that move crypto through Paiflow
Who	Students
Teams	10 teams of 1–4 people
Event starts	October 14, 2026, 10:30 AM PHT
Event development time	4 hours on October 14 to finish your app and prepare the demo
Prize pool	PHP 10,000 across five awards
Network	Stellar testnet
Online onboarding	October 12, 2026, 9:00 PM
Registration closes	October 11, 2026, 12:59 PM

Registration and resources

- **Register your team:** Google Form by October 11, 2026, 12:59 PM.
- **Starter repository:** GitHub.
- **AI context pack:** llms.md.
- **Sample project: Campus Snacks**
  - GitHub source
  - Live app
- **Support contact:** paiflow+jru.hack@artisam.xyz.

A maximum of 10 teams can participate. Solo teams are welcome, but teams are encouraged to have at least two members. If more than 10 teams apply, we will coordinate with the respective student organisation to determine the best way forward.

Resource content may be updated without prior notice.

Preparation and integration timeline

Starting now: choose a problem, talk to potential users and sketch your app.

Online onboarding: October 12, 2026, 9:00 PM. Hear the rules and watch a live demo: build, deploy, scan the QR and see the payout. The meeting link will be provided through **blockhub.academy** or through the student organisation representatives.

Before the event: build your app, deploy flows, connect Paiflow and test payments. Ask organisers for team credentials in advance; switch the starter to `team` when your deployment and token are ready.

October 14, 10:30 AM PHT: continue development, finish your integration and prepare the demo. Early deployment, API integration, wallet preparation and test payments are allowed.

Event development session: starts at **10:30 AM PHT on October 14**. Early integration is allowed. Follow the schedule below.

Start	Length	Block	What happens
10:30 AM	2 h	Development, part 1	Deploy and aim for your first confirmed payout
12:30 PM	1 h	Lunch + checkpoint	Optional coding; check your deployment and payout evidence
1:30 PM	2 h	Development, part 2	Complete integration and prepare your demo
3:00 PM	30 min	Submission	Submit before 3:30 PM with your deployment ID and payout transaction
3:30 PM	1 h	Pitching	10 teams × 5 minutes; random order drawn at 3:30 PM
4:30 PM	30 min	Deliberation + awards	

The challenge

Dev mode will stay disabled for the duration of the hackathon. Configure recipients and required values before deploying. Per-payment recipients, **Fill value via API** and **Dev values** are unavailable; standard Pay and Split flows remain available. `PAIFLOW_MODE=team` and `pnpm dev` do not enable flow Dev mode.

The theme is open. Start with a problem you or people around you have, then choose the Paiflow blocks that fit. These examples show how blocks combine and can inspire your own idea.

Paiflow deploys a flow you draw as Stellar contracts. Your app decides *who* pays and *why*; your flow decides *when* money moves and *where* it goes.

Idea	What it does	Paiflow pieces
Barkada bill splitter	A friend pays their share of a trip. The money automatically goes to friends who paid the group's expenses	`on_receive` + `split`, QR trigger page
Org dues and event tickets	QR payments for dues or tickets split between the org treasury and venue	`on_receive` + `split`, embed + live feed
Creator collab tip jar	Shared-project tips split between collaborators at agreed percentages	`on_receive` + `split`
Sari-sari store checkout	A customer scans and pays; the supplier's share is sent immediately	`on_receive` + `split` or `pay`
Digital baon	A parent sets up scheduled weekly allowance payments	`on_schedule` + `pay`
Tutor or club subscription	One configured member approves an allowance for recurring charges	`subscription` + allowance page
Freelance deadline escrow	Client pays up front; acceptance before the deadline pays the freelancer, otherwise the client is refunded	`on_receive` + `condition` (`time_before`) + two `pay` branches
Rewards from a Web2 app	A pre-funded pool pays a configured recipient when the app reports a completed goal	`web2_webhook` + authenticated callback + `pay` or `split`
Transparent donation drive	Donations split among beneficiaries, with a public feed showing payouts	`on_receive` + `split`, embed + live feed

Strong projects have a real user, a flow that fits their problem, and a demo with visible payouts.

On builds containing #577, public embeds show a read-only flow diagram and recent live activity, with a payment-page QR for supported payment and funding flows. If the live connection is unavailable, activity updates periodically. Webhook and streamer QR links add funds; they do not execute rewards or release streams. Subscription approval uses its separate allowance page. On builds containing #790, the configured customer opens that page and signs with their wallet without a Paiflow login or deployment token. For this hackathon, use ordinary subscriptions with the subscriber configured before deployment; Dev mode stays disabled throughout the event. If confirmation is unknown, check the existing approval status before approving again. Approval grants consent; the later scheduled charge must still be rehearsed with a real wallet and verified recipient payout. Confirm the deployed version with support before your rehearsal. For Web2 rewards, obtain the callback URL and secret from your authenticated owner deployment page and keep the secret on your server. Public embeds omit credentials and private configuration.

For setup instructions and example payment flows, see the AI context pack. Confirm feature availability with the support contact before choosing an idea.

Testnet funds have no real-money value. Never store wallet secret keys in your app.

Set up Paiflow once for the business, group or branch your app serves. For example, Campus Snacks represents one branch of a snack shop. Each customer pays separately, but every payment uses the same setup: 90% goes to the shop and 10% to the student organisation. You don’t need a new setup for each customer.

If your idea requires a separate payment setup for every customer, explain that as a future feature in your pitch.

On builds containing #911, Subscription apps can read current subscriber, deployed token, allowance and cancellation state through `GET /api/v1/deployments/{id}/subscription/state` using their deployment bearer token. Check `subscriptionState` in capabilities first; discovery contains metadata, while this endpoint reads live state. It does not authorize wallet approval or charge the customer. API response and error details include setup-

required Dev behavior, although Dev mode remains disabled for this hackathon. Hosted testnet verification of this endpoint is still pending; confirm the deployed version before using it in a rehearsal.

Subscription wallet controls (source implementation; hosted verification pending)

On builds containing #913, ordinary Subscription apps can request cancellation or reactivation through `subscription/cancel/{prepare,submit,reconcile}` and `subscription/reactivate/{prepare,submit,reconcile}` under the deployment-token API. Check capabilities and live Subscription state, then ask the configured subscriber's wallet to sign the exact prepared transaction. The token alone never consents. Cancellation revokes allowance; reactivation neither restores it nor immediately charges. Approval remains a separate wallet action. Persist the intent/hash and reconcile uncertainty before trying another preparation. Dev stays disabled for this hackathon. Hosted testnet verification is pending: confirm deployed availability and rehearse actual state/finality with support. The API reference includes the integration and recovery sequence.

Contract balance reads (source implementation; hosted verification pending)

Issue #915 adds `GET /api/v1/deployments/{id}/balances` for deployment-token apps. It reads funds held by supported deployed contracts, including Schedule funding and Webhook reward pools, without an owner session. Check the hosted capabilities and OpenAPI first: the source change still needs testnet chain-evidence validation before participant support is claimed.

Poll at most every 10 seconds. `tokenContract` identifies each current asset; `balanceStroops` stays an integer string. Inspect each node's `stale`/`readAt` and `freshness.status`/`totalsComplete` before displaying totals. Null or failed targets are unknown, not zero; partial totals omit failures. Multisig remains disabled and excluded. Contract balances are not Schedule payout availability or proof that a payment succeeded. Read the API balance contract for caching, unsupported kinds and retry behavior. The existing owner dashboard continues to use its separate balance endpoint.

Preview a deployed Swap

Issue #917 adds the source API `GET /api/v1/deployments/{id}/swap-quote?amountStroops=10000000`, authenticated with that deployment's `pfk_` bearer token. Hosted testnet comparison against a completed wallet-authorized swap remains pending; check the hosted OpenAPI before assuming availability. Use `operations.swapQuote.nodeIds` from discovery and provide `nodeId` if more than one Swap exists. Input is the amount reaching that particular node, not necessarily the amount deposited into the flow.

Assets, router and slippage come from the deployed contract, with no request overrides. Integer stroop output and minimum estimates include live read timestamps; `estimate:true` and `scope:"selected_node"` exclude guaranteed payouts and whole-flow predictions. `meetsMinimumAtQuote` only compares current estimates. A quote does not submit funds, authorize payment or replace final transaction simulation. See the API reference for freshness, limits and error handling.

Before the event

Choose your idea now. Build and integrate before the event; use October 14 to finish your app, verify its payout and prepare your demo.

- ☐ Register through the Google Form by October 11, 2026, 12:59 PM
- ☐ Attend onboarding on October 12, 2026, 9:00 PM; the meeting link will be provided through blockhub.academy or through the student organisation representatives. Share the instructions with your team

- ☐ Install Freighter on at least one team laptop and set it to testnet
- ☐ Install Node.js 22, pnpm 10.4.1 through Corepack, and Git; follow the Paiflow Guide setup instructions. Have a GitHub account
- ☐ Arrange your own internet connection for the event (for example, mobile data or a personal hotspot)
- ☐ Pick an idea

Internet connection: Participants must source their own internet connection for the event. Bring mobile data or a personal hotspot with enough data for development and payment testing.

October 12–13 preparation checklist:

- ☐ Install and run the supplied starter on your laptop
- ☐ Build your app, including its Paiflow integration and test payments once credentials are ready
- ☐ Prepare dedicated testnet wallets for your customer and payout recipients
- ☐ If using USDC, enable each recipient wallet to receive the exact testnet USDC asset by adding its trustline; follow the AI context pack's instructions
- ☐ Obtain testnet XLM for wallet setup and transaction fees, and testnet USDC for payments if needed

Testnet funding:

1. In Freighter, select **Testnet** and fund each new wallet with testnet XLM through Friendbot so the account exists and can cover reserves and fees. See Stellar's testnet setup guide. Friendbot supplies XLM, not USDC.
2. If using USDC, add a trustline in each customer and recipient wallet for code `USDC` and issuer `GBBD47IF6LWK7P7MDEVSCWR7DPUWV3NY3DTQEVFL4NAT4AQH3ZLLFLA5`. Have each wallet sign its own trustline transaction. A trustline to another issuer's USDC will not work, and adding one does not add funds.
3. Open Circle's testnet faucet, select **USDC** and **Stellar Testnet**, then enter the customer's public wallet address (`G…`). Confirm the USDC balance before testing payments. Repeat for any wallet that will fund a flow, and choose test amounts that fit the available balance.

For more setup instructions, see the AI context pack. If you need help with funding, contact support. Wallet and funding support will be available on site on October 14, with ready-to-use wallets as a last resort.

Organiser-provided wallets are for testnet only. Organisers retain copies of their keys. Never use these wallets for real funds or import their keys into a wallet you use for real money.

Your first payout

Get one confirmed payout working before polishing your app.

1. Receive your Paiflow credentials and log in to your team's account.
2. Check that all wallets have testnet XLM. For USDC payments, the customer also needs USDC and recipients must be set up to receive the exact asset.
3. Draw your payment flow in Paiflow using your team's recipient wallets. Replace default Alice/Bob/Charlie addresses; they are placeholders.
4. Deploy the flow, approve it in your wallet and wait for **CONFIRMED**.
5. Connect your app to that deployment using the AI context pack's setup instructions. Create an API token under **API access** and keep it on your app's server. Never put it in browser code or store wallet secret keys in your app.
6. Trigger a test payout using the instructions for your chosen flow in the AI context pack. For wallet-signed payments, approve it in Freighter and wait for **SUCCESS**. A pending or uncertain result is not a confirmed

failure: check its status and, for a wallet-signed payment, retry the same signed transaction rather than creating another payment.

7. Check that the recipients received the payout, then save its transaction hash for submission. You can verify it in the payment feed or a Stellar explorer.

Use your team's deployment for your qualifying payout. Payments through the starter's shared demo do not qualify; shared-demo access is also limited on shared Wi-Fi.

Rules

1. **Use Paiflow for the core money flow.** Your app's essential transfer must run through a Paiflow deployment.
2. **Testnet only.** No mainnet deployments or real funds.
3. **AI tools are encouraged.** Use any AI assistant; an AI context pack is provided.
4. **Build and integrate early.** Boilerplate, open-source libraries, the starter and AI tools are allowed. You may prepare wallets, deploy flows, connect your app to Paiflow, test payments and use the live feed before the event. Ask organisers for team credentials in advance; use your team deployment for qualifying evidence.
5. **Lunch coding is optional.** It does not count as extra time and carries no penalty.
6. **Respect shared infrastructure.** No load testing or spamming the Paiflow API or relayer. Rate limits apply.
7. **Organisers decide disputes.** Their ruling is final.

The beta app collects usage analytics (PostHog) to improve Paiflow.

Qualifying and submitting

To qualify, you need a confirmed Paiflow deployment and at least one on-chain payout from it. Otherwise, you may still pitch but won't be ranked or eligible for prizes.

Submit through the submission form by **3:30 PM PHT on October 14** (the end of integration part 2).

Judging

Each judge scores qualifying teams out of 100. Integration rewards fit to your problem over block count.

Criterion	What judges look at	Points	A top score looks like
Creativity (30)	Originality	15	A problem or angle judges haven't seen before
	Product experience	15	Paying or getting paid feels natural; the crypto part is invisible or delightful
Integration (40)	Flow fits the problem	15	Triggers, conditions and actions fit how money should move; nothing feels bolted on
	Paiflow integration	15	Paiflow pages or APIs are central to the app: QR trigger, embed, live feed, allowance page or `/api/v1`
	Works end to end	10	A live demo of your app's payment flow with a verifiable on-chain payout

Criterion	What judges look at	Points	A top score looks like
Real-world usefulness (30)	Real user, real problem	15	A specific user with a clear problem, ideally someone you've spoken to
	Could be used tomorrow	15	A plausible path to users in the Philippines, accounting for cost, trust and onboarding

Your final score and each criterion score are the averages across judges. Compare scores before rounding for display.

Under **Works end to end (10 points)**, a live demo can earn up to **10 points**; a recorded demo can earn up to **7 points**. Either must show your team's actual app, Paiflow deployment and a verifiable payout from it. The demo format affects only this criterion; all other criteria keep the same maximum points. The qualifying on-chain payout remains mandatory.

Pitching

Each team gets **5 minutes: 3 for a pitch and demo, then 2 for judges' Q&A.**

- Pitch and demo your app's payment flow live or in a screen recording, and show the payout in the live feed, recipient balance or explorer transaction.
- At 3:00, your pitch ends and Q&A begins. The timer is strict.
- Pitch order is drawn at random when development ends.
- Set up during the previous team's Q&A.
- You may use a screen recording, including if your live demo fails. If the recording provides the successful demonstration, the recorded-demo maximum of 7 points applies to **Works end to end**.

Prizes

PHP 10,000 across five awards, for qualifying teams only. Prizes go by QR Ph to the member named in your submission.

Award	Prize (PHP)	How it's decided
1st place	4,000	Highest score
2nd place	2,500	Second-highest score
3rd place	1,500	Third-highest score
Best Integration	1,000	Highest Integration score among eligible teams
Best Real-World Usefulness	1,000	Highest Real-world usefulness score among eligible teams

If fewer than three teams qualify: award only the available places at their published amounts. With one qualifying team, award first place only; with two, award first and second. Both special awards are still awarded when at least one team qualifies. Vacant placing prizes remain unawarded, and unused prize funds are not redistributed. If no team qualifies, no prizes are awarded.

Winning projects are featured on the Paiflow homepage and socials.

Deployment funding (#916)

Schedule top-ups and Webhook pool deposits use deployment-token GET/POST `/api/v1/deployments/{id}/funding`, `/funding/submit` and `/funding/reconcile`. Any consenting funding wallet may sign; a token alone cannot move funds. Funding adds funds only and does not execute payouts or callbacks. One unresolved intent per node blocks replacements until confirmed finality or ledger-proven expiry; UNKNOWN never permits another deposit. Public `/trigger`, `/submit-trigger` and hosted funding remain available. Webhook callbacks separately require `x-webhook-secret`. See the complete signing and recovery example. Hosted testnet verification remains pending.

Schedule integration APIs

The Schedule state and controller wallet prepare/sign/submit/reconcile APIs are implemented under `/api/v1/deployments/{id}/schedule`. Hosted testnet verification remains pending before they are advertised as participant-ready. Use capabilities `scheduleState` and `scheduleControls`, then current state and simulated preparation to check eligibility. Pause/resume/retrieval need the deployed controller's wallet consent; a deployment token is API access only. Retrieval requires a paused permitted Schedule and returns unvested funds to that controller, not scheduled recipients. Holding balance differs from vested payout availability. Never prepare another envelope merely because a request times out. Follow the complete Schedule wallet and recovery example.